

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 04 -

RODANDO A PRIMEIRA APLICAÇÃO: PODS E DEPLOYMENTS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

O QUE VEM POR AÍ?

Nesta aula, vamos iniciar os primeiros passos práticos com Kubernetes, entendendo como executar aplicações por meio de Pods e Deployments. Você conhecerá os conceitos essenciais da menor unidade executável no Kubernetes (o Pod) e verá como gerenciar aplicações de maneira escalável e confiável utilizando Deployments.

Partiremos da criação de Pods básicos até chegar a estratégias mais avançadas de deployment, como rolling updates, rollbacks e scaling. O objetivo é oferecer uma experiência prática que permita a você rodar suas primeiras aplicações em um cluster Kubernetes, compreendendo as diferenças entre a execução direta de Pods e o uso de controllers como os Deployments.

Além disso, vamos analisar como esses objetos se relacionam com os componentes do Control Plane e dos Worker Nodes, destacando o papel do API Server, do Scheduler e do kubelet no processo de colocar suas aplicações em funcionamento no cluster.

HANDS ON

As videoaulas práticas terão como foco a criação, execução e administração de Pods e Deployments. Por meio de atividades hands on, você acompanhará todo o ciclo de vida de uma aplicação no Kubernetes, desde a definição dos manifestos em YAML até a solução de problemas comuns no ambiente.

Durante as práticas, o aluno poderá aprender a:

- Criar e administrar Pods individuais;
- Definir e aplicar Deployments;
- Realizar o scaling horizontal de aplicações;
- Executar rolling updates e rollbacks;
- Configurar resource limits e requests;
- Utilizar labels e selectors para organização;
- Monitorar o status e analisar os logs das aplicações.

Além disso, serão abordados cenários reais de operação, como:

- Debug de Pods em falha;
- Estratégias de deployment para diferentes ambientes;
- Configuração de health checks (liveness e readiness probes);
- Gerenciamento de configurações e secrets.

SAIBA MAIS

Rodando a Primeira Aplicação: Pods e Deployments no Kubernetes

Para executar aplicações no Kubernetes, é fundamental entender dois elementos básicos: Pods, que representam a menor unidade executável, e Deployments, que funcionam como controllers responsáveis por gerenciar Pods de maneira inteligente e automatizada. Esses recursos interagem diretamente com a arquitetura do Control Plane e dos Worker Nodes, possibilitando que suas aplicações sejam efetivamente executadas no cluster.

Pods: A Unidade Fundamental de Execução

Um Pod consiste em um ou mais contêineres que compartilham rede e armazenamento. Ele representa a menor entidade que pode ser criada, implantada e administrada dentro do Kubernetes.

Anatomia de um Pod – Principais características:

- **IP único:** todos os contêineres do Pod utilizam o mesmo endereço IP.
- **Volumes compartilhados:** permitem acesso comum ao armazenamento entre os contêineres.
- **Ciclo de vida conjunto:** os contêineres são iniciados e encerrados em sincronia.
- **Comunicação via localhost:** os contêineres se comunicam entre si utilizando localhost.

Estrutura básica:

```
apiVersion: v1
kind: Pod
metadata:
  name: meu-primeiro-pod
  labels:
spec:
  containers:
  - name: nginx-container
    image: nginx:1.21
```

```
ports:  
- containerPort: 80
```

Interação com a Arquitetura do Kubernetes

Fluxo de criação de um Pod:

1. O kubectl envia a requisição para o API Server.
2. O API Server valida a solicitação e armazena os dados no etcd.
3. O Scheduler escolhe o nó mais adequado com base nos recursos disponíveis.
4. O kubelet do Worker Node recebe as instruções de execução.
5. O Container Runtime faz o download da imagem e inicia o contêiner.

Estados Possíveis de um Pod

- **Pending:** aguardando agendamento ou download da imagem.
- **Running:** pelo menos um contêiner está em execução.
- **Succeeded:** todos os contêineres finalizaram com sucesso.
- **Failed:** um ou mais contêineres falharam.
- **Unknown:** o estado não pôde ser determinado.

Deployments: Controle Inteligente de Pods

Um Deployment é um controller de alto nível que administra ReplicaSets, responsáveis por gerenciar os Pods. Ele oferece recursos avançados para garantir estabilidade e flexibilidade, como rolling updates, rollbacks e scaling de forma declarativa.

Por que preferir Deployments em vez de criar Pods diretamente?

Principais vantagens dos Deployments:

- **Alta disponibilidade:** garante sempre a quantidade desejada de réplicas.
- **Rolling updates:** permite atualizar versões sem gerar indisponibilidade.
- **Rollback automático:** reverte para a versão anterior em caso de falhas.

- **Self-healing:** recria Pods que apresentarem problemas.
- **Scaling simplificado:** ajusta facilmente o número de réplicas conforme a demanda.

Estrutura de um Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
          ports:
            - containerPort: 80
          resources:
            requests:
              memory: "64Mi"
              cpu: "250m"
            limits:
              memory: "128Mi"
              cpu: "500m"
```

Controller Manager e Deployments

O Controller Manager é responsável por executar o Deployment Controller, que tem as seguintes funções:

- Monitorar continuamente o estado desejado em comparação com o estado atual.
- Criar ou remover ReplicaSets conforme a necessidade.

- Administrar todo o processo de rolling update.
- Implementar diferentes estratégias de deployment.

Estratégias de Deployment

Rolling Update: Realiza substituição gradual de Pods, preservando a disponibilidade.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
```

Processo:

1. Cria novo ReplicaSet com nova versão;
2. Escala novo ReplicaSet gradualmente;
3. Escala antigo ReplicaSet para baixo;
4. Mantém número total próximo ao desejado.

Recreate: Remove todos os Pods antigos antes de criar novos:

```
spec:
  strategy:
    type: Recreate
```

Quando usar: aplicações que não suportam múltiplas versões simultâneas.

Resource Management

Requests vs Limits:

Requests (Garantia mínima):

- CPU e memória reservados pelo Scheduler;
- Influencia decisão de agendamento;

- Base para billing em clouds;

Limits (Teto máximo):

- Previne consumo excessivo de recursos;
- Protege nós de resource starvation;
- Trigger para OOMKill se exceder memória.

Health Checks e Probes

Liveness Probe: Determina se contêiner está healthy:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 1
```

Readiness Probe: Determina se Pod está pronto para receber tráfego:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds:
  periodSeconds: 5
```

Startup Probe: Para aplicações com inicialização lenta:

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  periodSeconds: 10
  failureThreshold: 30
```

Labels e Selectors

Organização com Labels: Labels são pares chave-valor para identificar e agrupar objetos:

```
metadata:
  labels:
    app: nginx
    version: v1.21
    environment: production
    tier: frontend
```

Selectors em Deployments: conectam Deployments aos Pods gerenciados:

```
selector:
  matchLabels:
    app: nginx
  matchExpressions:
    - key: version
      operator: In
  values: [v1.21, v1.22]
```

Comandos kubectl Essenciais

Criando um pod: `kubectl create -f pod.yaml` `kubectl run nginx --image=nginx.`

Listando Pods: `kubectl get pods` `kubectl get pods -o wide.`

Criando um Deployment: `kubectl create deployment nginx --image=nginx`
`kubectl apply -f deployment.yaml.`

Listando um Deployment: `kubectl get deployments` `kubectl get deploy.`

Escalando um Deployment: `kubectl scale deployment nginx --replicas=5.`

Rolling Update: `kubectl set image deployment/nginx nginx=nginx:1.22.`

Rollback: `kubectl rollout undo deployment/nginx` `kubectl rollout history deployment/nginx.`

Status do Rollout: `kubectl rollout status deployment/nginx.`

MERCADO, CASES E TENDÊNCIAS

Casos de uso reais:

Airbnb: opera milhares de microsserviços utilizando Deployments com estratégias avançadas, como canary e blue-green, para lidar com milhões de requisições diárias.

Spotify: executa jobs massivos de processamento de dados musicais em Pods, com Deployments automatizados via práticas de GitOps, suportando mais de 300 squads de desenvolvimento.

Pinterest: adota Deployments com políticas rígidas de resource quotas para otimizar custos, rodando workloads de machine learning e processamento de imagens em Pods especializados.

Tendências atuais:

Serverless Pods: uso do Knative para execução sob demanda.

Multi-cloud Deployments: federação de clusters em diferentes provedores.

GitOps: deployments declarativos com ferramentas como ArgoCD e Flux.

Observabilidade: integração de service mesh (ex.: Istio) para telemetria avançada.

Segurança: aplicação de Pod Security Standards e admission controllers.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos os conceitos fundamentais para executar aplicações no Kubernetes através de Pods e Deployments. Você compreendeu que:

Pods constituem a menor unidade executável no Kubernetes, podendo abranger um ou múltiplos contêineres que compartilham rede, armazenamento e ciclo de vida.

Deployments funcionam como controllers de alto nível, administrando o ciclo de vida dos Pods de forma automatizada, assegurando alta disponibilidade, escalabilidade e autorrecuperação.

Principais tópicos examinados:

Implementação e administração de Pods: estrutura básica em YAML, estados possíveis (Pending, Running, Failed, etc.) e interação com os componentes do Kubernetes (API Server, Scheduler, kubelet).

Benefícios dos Deployments: rolling updates, rollbacks automáticos, scaling simplificado e garantia de réplicas.

Estratégias de deployment: RollingUpdate (atualização gradual) e Recreate (substituição completa).

Administração de recursos: requests (garantia mínima) e limits (teto máximo) para CPU e memória.

Health checks: emprego de liveness, readiness e startup probes para verificar a saúde dos contêineres.

Organização com labels e selectors: como identificar e agrupar recursos de forma eficiente.

Comandos práticos do kubectl: desde a implementação e listagem de Pods e Deployments até operações de scaling, atualização e rollback.

REFERÊNCIAS

Configure Liveness, Readiness and Startup Probes. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>. Acesso em: 03 set. 2025.

Deployments. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>. Acesso em: 03 set. 2025

Managing Resources for Containers. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>. Acesso em: 03 set. 2025.

Pods. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/workloads/pods/>. Acesso em: 03 set. 2025.

PALAVRAS-CHAVE

Pods. Deployments. Rolling Updates.

EMANIP



POSTECH